EC GADGET REPAIR SHOP: SIMPLE REPAIR TRACKING SYSTEM

Introduction To Computing Project
Presented to the
Faculty of the College of Information Technology
Cagayan de Oro College
Cagayan de Oro City

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Information Technology

NAME OF RESEARCHER

DICO, MARY GUILLIAN S.
LACAY, MARK LAWRENCE T.
MALDO, VILLE KYLE E.
PADUA, LEONARDO MIGUEL N.
PAYLA, JHON FRIX D.
QUINTERO, HURL E.

OCTOBER, 2026

1.1 Introduction

Information management systems help organizations organize and manage important records needed for daily operations. Management information systems can support organizations in making decisions by providing information needed for administrative activities (Santoso et al., 2022). Proper records management is also important because maintaining accurate business records can support business operations, decision-making, and organizational effectiveness (Mintah et al., 2022). This is relevant to gadget repair shops because staff need to manage customer information, gadget details, repair problems, costs, and repair status in an organized manner.

The EC Gadget Repair Shop in Barra, Opol, Misamis Oriental provides gadget repair services and is served by staff and technicians who handle customer and repair information. Based on the proponents' interview with the shop, the current process involves recording and managing repair information for different customer transactions. The interview identified three problems that the project addresses: difficulty in recording and organizing repair information, difficulty in tracking repair status, and difficulty in managing multiple repair transactions. These problems involve keeping customer and gadget details organized, determining whether a gadget is waiting for repair, being repaired, or completed, and managing several repair records at the same time. Similar difficulties have also been reported in repair-service settings that rely on manual records, where information may be difficult to manage or become misplaced (May & Aman, 2021).

To address these problems, the proponents developed the Simple Repair Tracking System for EC Gadget Repair Shop using a Python console-based application. The system provides separate functions for staff and customers. Customers can register an account, log in, view their repair records, monitor repair status, and access their repair receipts. Staff can log in using the provided staff account, select registered customers, record gadget brands and models, enter repair problems and their corresponding costs, view repair records, update repair statuses, generate receipts, and remove repair records when necessary. Through these functions, the system aims to organize repair information, support repair-status monitoring, and help staff manage multiple repair transactions more efficiently.

1.2 Statement of the Problem

General Problem

The EC Gadget Repair Shop faces challenges in managing repair information and transactions through its existing process. These challenges can make it difficult for staff to organize repair records, track repair status, and manage multiple customer transactions efficiently. There is a need for a system that organizes repair information and allows customers to access their own repair records and receipts.

Specific Problems

1. Difficulty in Recording and Organizing Repair Information.

Recording customer names, gadget details, repair problems, and repair costs can make it difficult to keep repair information accurate and organized.

2. Difficulty in Tracking Repair Status.

Monitoring whether a gadget is waiting for repair, being repaired, or completed can make it difficult for customers and staff to determine its current repair status.

3. Difficulty in Managing Multiple Repair Transactions.

Handling several customer repair transactions can make it harder to identify and manage the correct repair record for each customer and provide access to the corresponding repair receipt.

1.3 Objectives of the Study

General Objective

To develop the Simple Repair Tracking System for EC Gadget Repair Shop, a Python console-based application that enables staff to manage repair records and transactions while allowing customers to access their own repair information and receipts.

Specific Objectives

1. To record and organize repair information by allowing staff to enter and view customer names, gadget details, repair problems, and corresponding repair costs.

2. To track repair status by allowing staff to update repair statuses and customers to view the current status of their repair records.

3. To manage multiple repair transactions by allowing staff to view organized repair records and generate receipts, while enabling customers to view their own repair records and receipts.

1.4 Scope

The Simple Repair Tracking System for EC Gadget Repair Shop is a Python console-based application developed to help staff manage gadget repair transactions and allow customers to access their own repair information. The system provides separate functions for staff and customers. Customers can register an account using their full name, username, and password, log in to their accounts, view their repair records, check repair statuses, and access their repair receipts. Staff can log in using the provided staff account to manage customer repair transactions.

The system allows staff to select registered customers, record gadget brands and models, enter multiple repair problems and their corresponding costs, calculate the total repair cost, and create repair records with receipt numbers and transaction dates. Staff can view all repair records, update repair statuses to Waiting for repair, Being repaired, or Completed, generate receipts for selected transactions, and remove repair records when necessary. The system validates certain inputs, such as repair costs and selected repair records, to help prevent invalid entries. All account and repair information is stored temporarily in the program's memory while the system is running. The application operates through a text-based console on a local computer and is intended for one user at a time.

1.5 Limitations

The Simple Repair Tracking System for EC Gadget Repair Shop is limited to a Python console-based application that operates on a local computer. Customer account information and repair records are stored temporarily in the program's memory and are lost when the program is closed. As a result, customers and staff must register accounts and enter repair information again when the program is restarted. The system does not use a database or permanent file storage to preserve records.

The system is intended for one user at a time and does not support simultaneous access from multiple computers. It does not include online payment processing, SMS or email notifications, inventory management, accounting functions, or a web-based or mobile application. In addition, the system uses a predefined staff account and stores passwords as plain text, making its account system unsuitable for handling sensitive information in a real business environment. Therefore, the system is intended as an educational project for demonstrating basic repair tracking and transaction management rather than as a fully secure, production-ready application.

1.6 System Requirements

Hardware Requirements

- **Processor:** Intel Core i3 or equivalent
- **RAM:** Minimum 4GB
- **Storage:** At least 1GB of available storage
- **Display:** Monitor or Laptop display
- **Input Devices:** Keyboard and mouse
- **Network:** None Required

Software Requirements

- **Operating System:** Windows or another compatible operating system.
- **Python Version:** Python 3.14.8
- **Required Packages:** Python Standard Library only
- **File/Database Requirements:** None .
- **Development Environment:** Visual Studio Code.

Staff Functional Requirements

- Staff can log in using the predefined staff account.
- Staff can select registered customers when creating repair records.
- Staff can record gadget brands and models, multiple repair problems, and their corresponding costs.
- Staff can view all repair records and their current statuses.
- Staff can update repair statuses to Waiting for repair, Being repaired, or Completed.
- Staff can generate receipts for selected repair transactions.
- Staff can remove repair records after confirmation.

Customers Functional Requirements

- Customers can register an account using their full name, username, and password.
- Customers can log in using their registered credentials.
- Customers can view their own repair records and current repair statuses.
- Customers can view their own repair receipts.

1.7 Code Snippet of Simple Repair Tracking System for Gadget Repair Shop

Snippet 1 – Adding and Viewing Repair Records

Objective 1: To record and organize repair information by allowing staff to enter and view customer names, gadget details, repair problems, and corresponding repair costs.

```
repair = {
    "receipt_no": next_receipt_no,
    "customer_username": customer["username"],
    "name": customer["full_name"],
    "gadget": gadget,
    "problem": problem_text,
    "cost": total_cost,
    "status": "Waiting for repair",
    "date_time": date_time,
    "completed_time": ""
}

repairs.append(repair)

def view_repairs():
    print("\n--- VIEW ALL REPAIRS ---")

    if not repairs:
        print("No repair records found.")
        pause()
        return

    print("\nReceipt No. | Customer | Gadget | Status")
    print("----------------------------------------")

    for repair in repairs:
        print(
            f"{repair['receipt_no']} | "
            f"{repair['name']} | "
            f"{repair['gadget']} | "
            f"{repair['status']}"
        )

    while True:
        choice = input(
            "\nEnter receipt number to view full details, or Q to go back: "
        ).strip().lower()

        if choice == "q":
            break

        try:
            receipt_no = int(choice)
```

```
            repair = find_repair_by_receipt(receipt_no)

            if repair is None:
                print("Receipt number not found. Please try again.")
                continue

            print("\n--- FULL REPAIR INFORMATION ---")
            print("Receipt Number:", repair["receipt_no"])
            print("Customer:", repair["name"])
            print("Gadget:", repair["gadget"])
            print("\nRepair Problems:")
            print(repair["problem"])
            print("Total Cost: ₱", repair["cost"])
            print("Status:", repair["status"])
            print("Date & Time Added:", repair["date_time"])

            if repair["completed_time"]:
                print(
                    "Completed Date & Time:",
                    repair["completed_time"]
                )

            pause()

        except ValueError:
            print("Invalid choice. Enter a receipt number or Q.")
```

The code stores each repair transaction in a dictionary containing the receipt number, customer information, gadget details, repair problems, cost, status, and date and time. The repairs.append(repair) statement adds the new repair record to the list, while the view_repairs() function displays all recorded repairs and allows staff to enter a receipt number to view the complete details. This supports Objective 1 by helping staff record, organize, and retrieve repair information more efficiently.

Snippet 2 – Updating and Tracking Repair Status

Objective 2: To track repair status by allowing staff to update repair statuses and customers to view the current status of their repair records.

```
def update_status():
    print("\n--- UPDATE REPAIR STATUS ---")

    repair = choose_repair(
        "Enter receipt number to update: "
    )

    if repair is None:
        pause()
        return

    print("\n1. Waiting for repair")
    print("2. Being repaired")
    print("3. Completed")

    while True:
        choice = input("Select new status: ").strip()

        if choice in ["1", "2", "3"]:
            repair["status"] = STATUSES[int(choice) - 1]
            break

        print("Invalid choice. Select 1, 2, or 3.")

    if repair["status"] == "Completed":
        repair["completed_time"] = datetime.now().strftime(
            "%B %d, %Y - %I:%M %p"
        )
    else:
        repair["completed_time"] = ""

    print("\nRepair status updated successfully!")
    pause()
```

The update_status() function allows staff to select a repair record using its receipt number and change its status to Waiting for repair, Being repaired, or Completed. It validates the selected option and records the completion date and time when a repair is marked Completed. Customers can check their current repair status through the separate customer_view_repairs() function. This supports the second objective by helping staff track repair progress and allowing customers to view their repair records.

Snippet 3 – Managing Repair Transactions and Generating Receipts

Objective 3: To manage multiple repair transactions by allowing staff to view organized repair records and generate receipts, while enabling customers to view their own repair records and receipts.

```
def generate_receipt(repair=None):
    print("\n--- GENERATE RECEIPT ---")

    if repair is None:
        repair = choose_repair("Enter receipt number: ")

    if repair is None:
        pause()
        return

    print("\n========== GADGETFIX RECEIPT ==========")
    print("Receipt Number:", repair["receipt_no"])
    print("Customer Name:", repair["name"])
    print("Gadget:", repair["gadget"])
    print("--------------------------------------")
    print("Repair Problems:")
    print(repair["problem"])
    print("--------------------------------------")
    print("Total Repair Cost: ₱", repair["cost"])
    print("Status:", repair["status"])
    print("Date & Time Added:", repair["date_time"])

    if repair["completed_time"]:
        print(
            "Completed Date & Time:",
            repair["completed_time"]
        )

    print("======================================")
    pause()


def customer_receipt(account):
    print("\n--- VIEW MY RECEIPT ---")

    my_repairs = [
        repair for repair in repairs
        if repair["customer_username"].lower()
        == account["username"].lower()
    ]

    if not my_repairs:
        print("You have no repair records yet.")
        pause()
        return
```

```
for repair in my_repairs:
    print(
        f"Receipt No: {repair['receipt_no']} | "
        f"Gadget: {repair['gadget']} | "
        f"Status: {repair['status']}"
    )

while True:
    try:
        receipt_no = int(
            input("Enter your receipt number: ")
        )

        repair = find_repair_by_receipt(receipt_no)

        if (
            repair is not None
            and repair["customer_username"].lower()
            == account["username"].lower()
        ):
            generate_receipt(repair)
            return

        print(
            "That receipt was not found under your account."
        )

    except ValueError:
        print("Enter a whole-number receipt number.")
```

The generate_receipt() function displays the receipt number, customer name, gadget details, repair problems, total cost, status, and relevant dates. The customer_receipt() function allows customers to view their own repair records and access a receipt after verifying that it belongs to their account. These functions support Objective 3 by helping staff generate receipts and allowing customers to access their own repair information.

References

May, T. P., & Aman, H. (2021). A development of a management information system for home appliance repairing. *Applied Information Technology and Computer Science, 2(2)*, 1650–1662. https://doi.org/10.30880/aitcs.2021.02.02.107

Mintah, C., Gabir, M., Aloo, F., & Ofori, E. K. (2022). Do business records management affect business growth? *PLOS ONE, 17(3)*, e0264135. https://doi.org/10.1371/journal.pone.0264135

Santoso, B., Hikmawan, T., & Imaniyati, N. (2022). Management information systems: Bibliometric analysis and its effect on decision making. *Indonesian Journal of Science and Technology, 7(3)*, 583–602. https://doi.org/10.17509/ijost.v7i3.56368